



# Data Lakehouse

## Iceberg

Open Table Format (OTF)

[www.3aayaam.in](http://www.3aayaam.in)

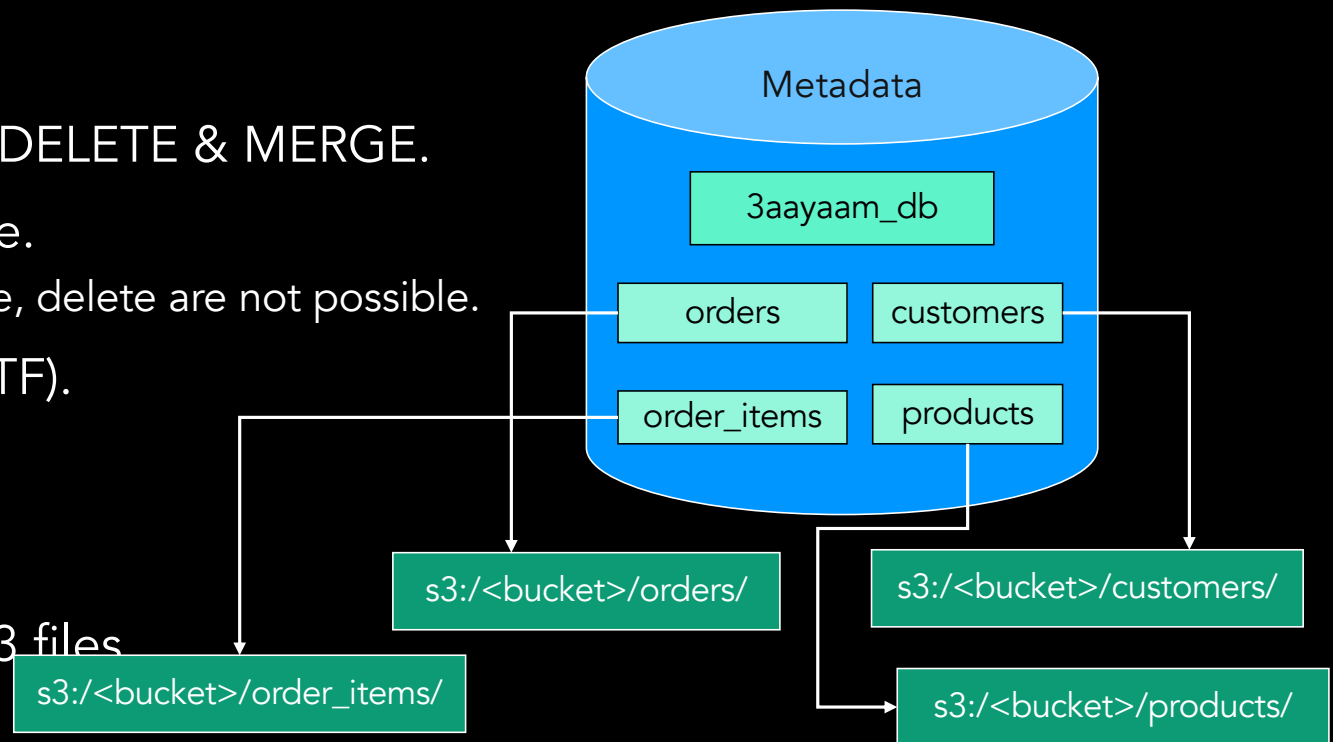
- Lakehouse
- Iceberg Architecture
- EMR with Iceberg
- Jupyter Notebook with Iceberg
- Iceberg Table Operations
- Optimize & Vacuum
- Time Travel
- Hidden Partitions
- Medallion Architecture
- Lakehouse Architecture

# Agenda

[www.3aayaam.in](http://www.3aayaam.in)

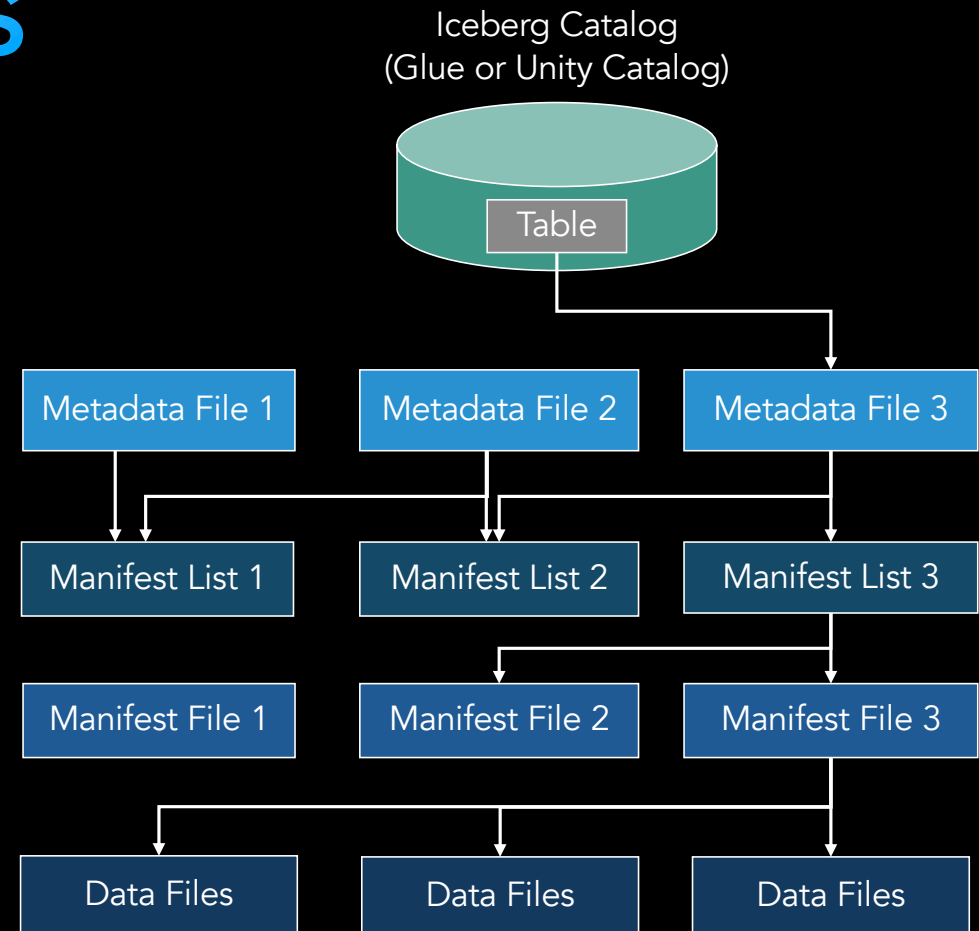
# Lakehouse

- Storage – Distributed (S3 & HDFS)
- Data Lake – S3/HDFS.
- No row level UPDATE, DELETE & MERGE.
- Limitations of Data Lake.
  - Single row update, merge, delete are not possible.
- Open Table Format (OTF).
  - Iceberg
  - HUDI
  - Delta Lake
- Glue Data Catalog & S3 files

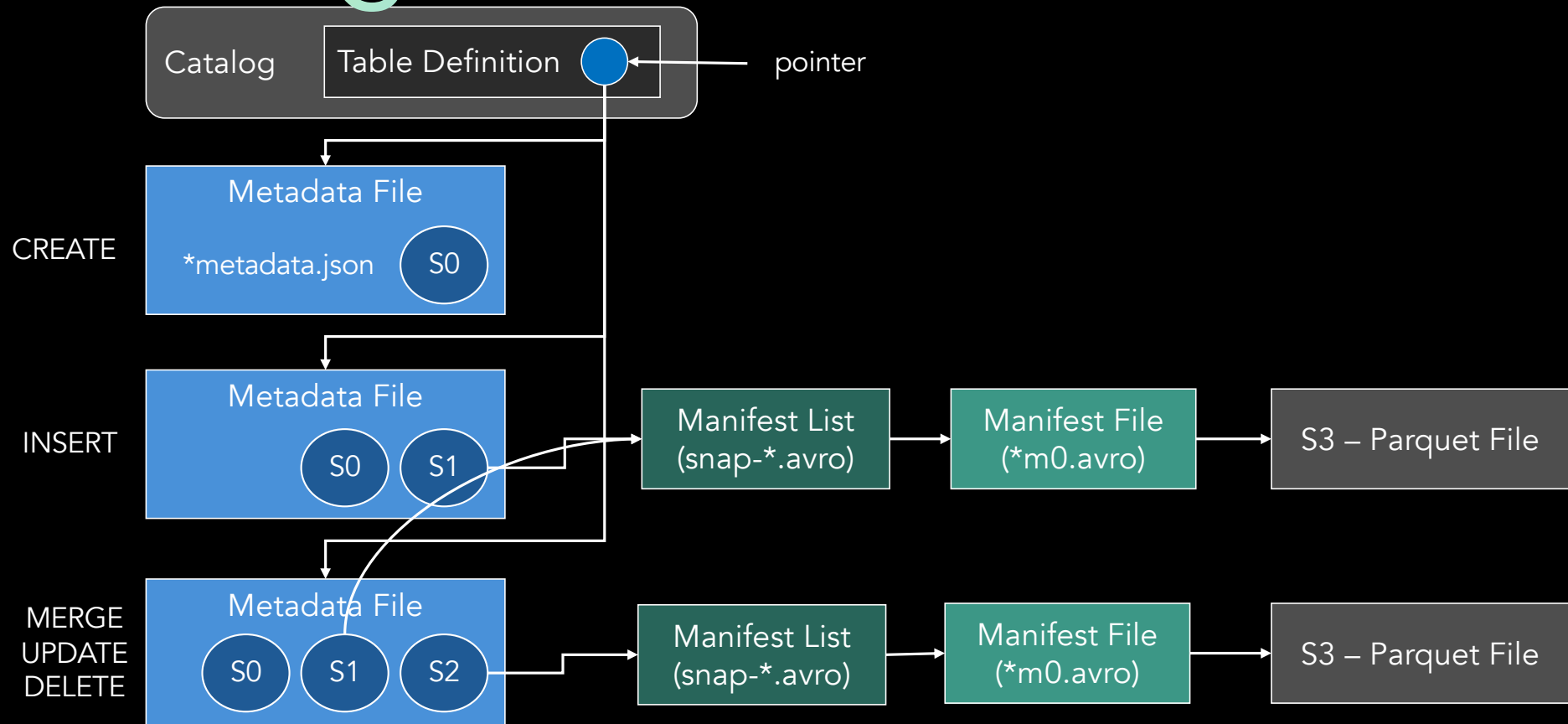


# Iceberg Tables

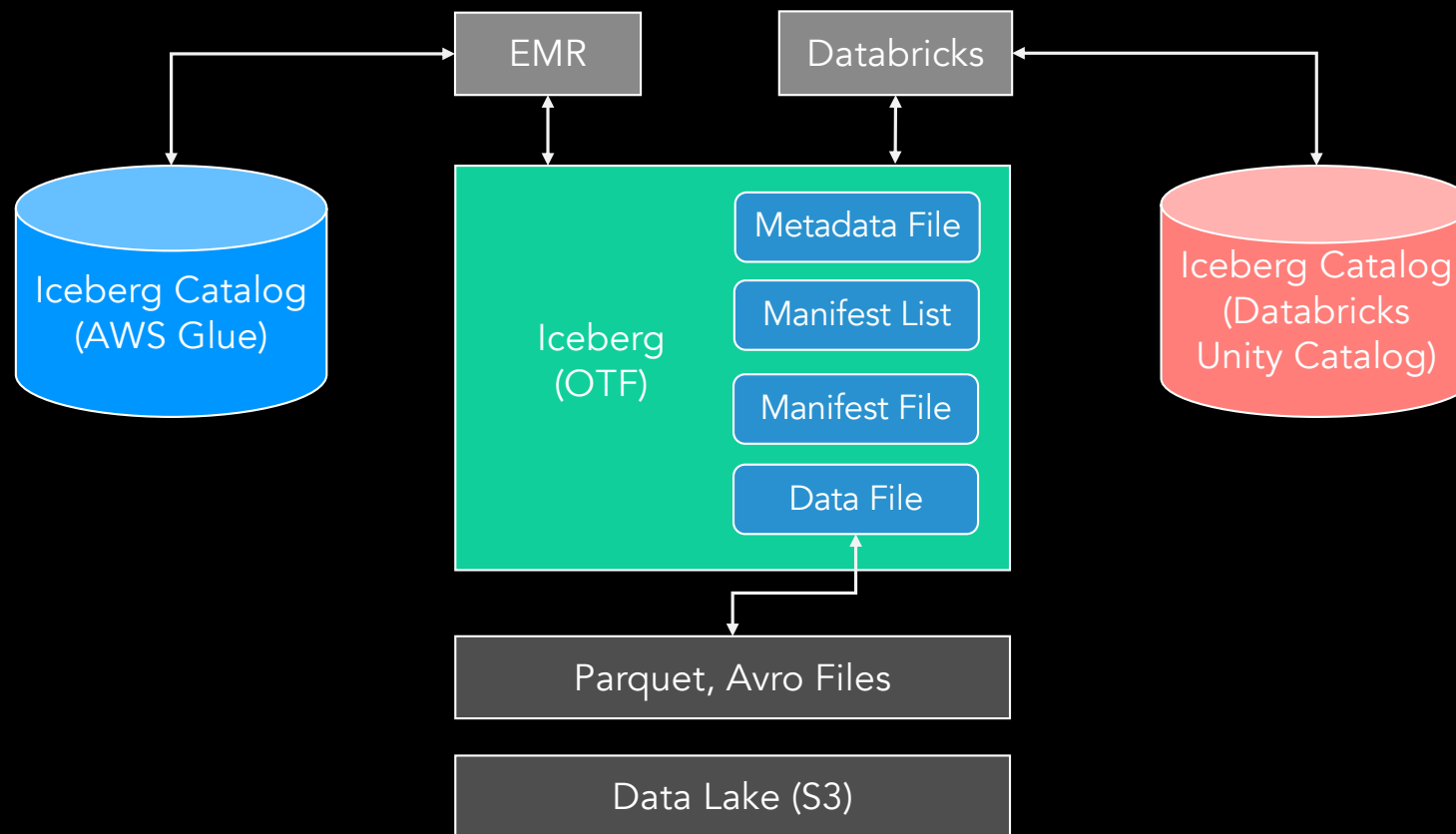
- Provides metadata structure to enable row level changes in Data Lake.
- Lakehouse = Data Lake + OTF
- Architecture:
  - Iceberg Catalog
  - Metadata File – Contains info about Table's schema, partition and snapshots. `*.metadata.json`
  - Manifest List – Contains list of manifest files. Info about manifest files that make up a snapshot. `snap-*.avro`
  - Manifest File – Information about data files. `*m0.avro`
  - Data File – Contains data along with delete files.



# Iceberg Architecture



# Lakehouse Architecture



# EMR Cluster with Iceberg

- While creating EMR cluster.

```
[
  {
    "Classification": "iceberg-defaults",
    "Properties": { "iceberg.enabled": "true" }
  }
]
```

- While executing Spark application using spark-submit.

```
spark-submit ... --conf "spark.jars=/usr/share/aws/iceberg/lib/iceberg-spark3-runtime.jar"
```

# EMR Jupyter Notebook with Iceberg

- Provide configuration parameters in the notebook using %%configure.

```
%%configure -f
{
  "conf": {
    "spark.sql.catalog.my_catalog" : "org.apache.iceberg.spark.SparkCatalog",
    "spark.sql.catalog.my_catalog.type" : "glue",
    "spark.sql.catalog.my_catalog.warehouse" : "s3://<bucket>/<folder>/",
    "spark.sql.defaultCatalog" : "my_catalog",
    "spark.sql.extensions" : "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions"
  }
}
```

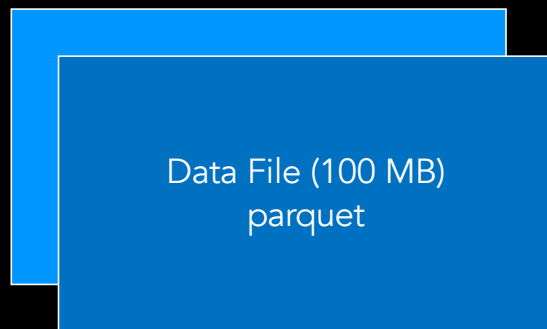


# Iceberg Table Operations

- CREATE TABLE ... USING ICEBERG LOCATION "<s3-path>"
- create table ... using iceberg location "<s3-path>" partitioned by (col1, col2, ...)
- create table ... using iceberg AS SELECT ...
- DROP TABLE ...
- ALTER TABLE ... WRITE ORDERED BY (col1, col2, ...)
- ALTER TABLE ... RENAME TO ...
- alter table ... add column ...
- alter table ... alter column ...
- alter table ... rename column ...
- alter table ... drop column ...
- alter table ... add/drop partition ...

# Delete Files

- Delete files – Files that contain only the changes.
- Copy-On-Write (COW) & Merge-On-Read (MOR)



COW



MOR

Delete File (parquet)

Delete File (parquet)

Delete File (parquet)

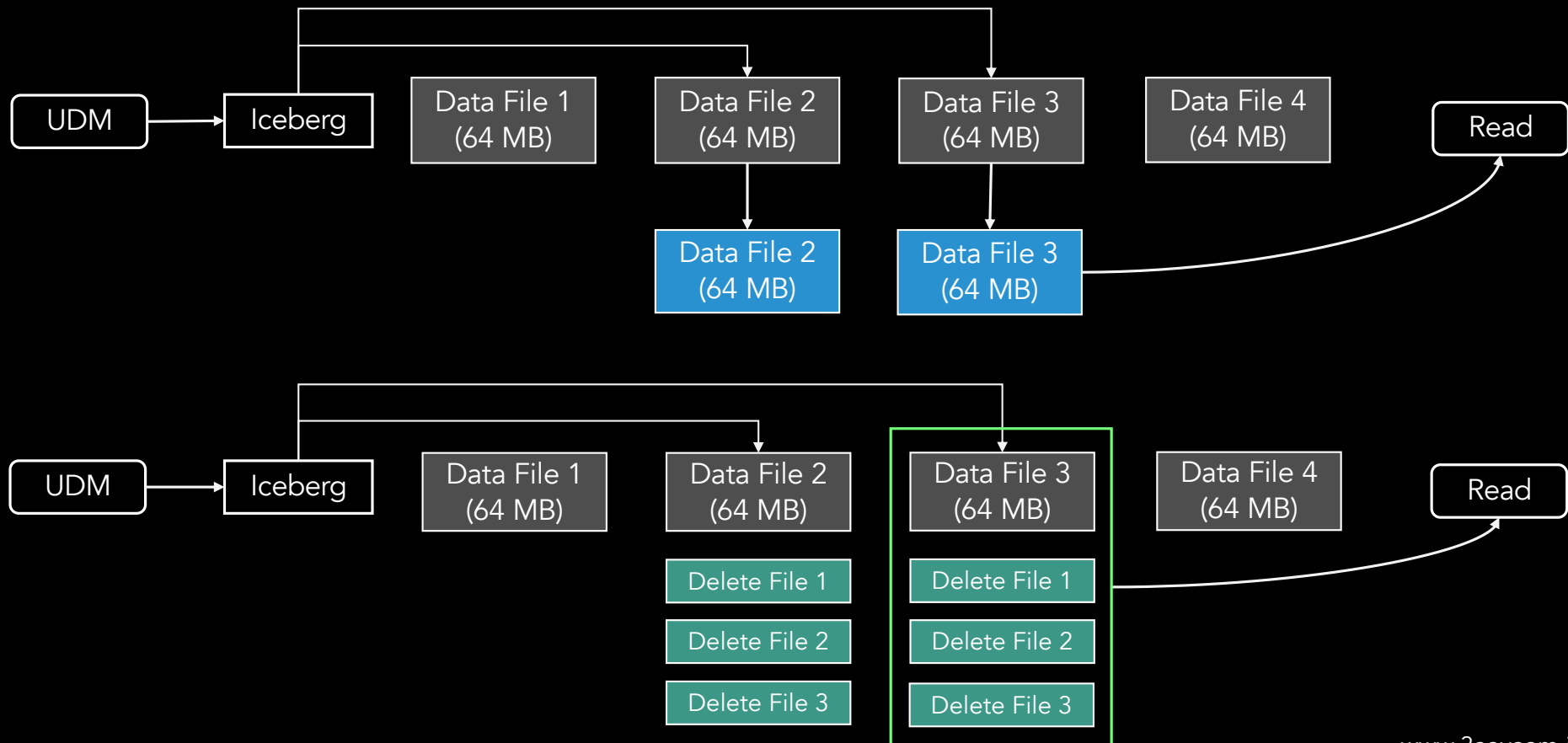
# Iceberg Table Properties

- CREATE TABLE ... USING ICEBERG **TBLPROPERTIES** ('key' = 'value')
- read.parquet.vectorization.enabled – true
- read.parquet.vectorization.batch-size – 5000
- write.format.default – Default file format for Iceberg tables. Parquet, avro, orc.
- write.delete.format.default – Default Delete file format. Parquet, avro, orc.
- write.delete.target-file-size-bytes – Delete file size. 64 MB.
- write.delete.mode– COW or MOR. copy-on-write or merge-on-read. COW is default.
- write.update.mode – COW or MOR. copy-on-write or merge-on-read. COW is default.
- write.merge.mode – COW or MOR. copy-on-write or merge-on-read. COW is default.
- write.delete/update/merge.isolation-level – Serializable or Snapshot.

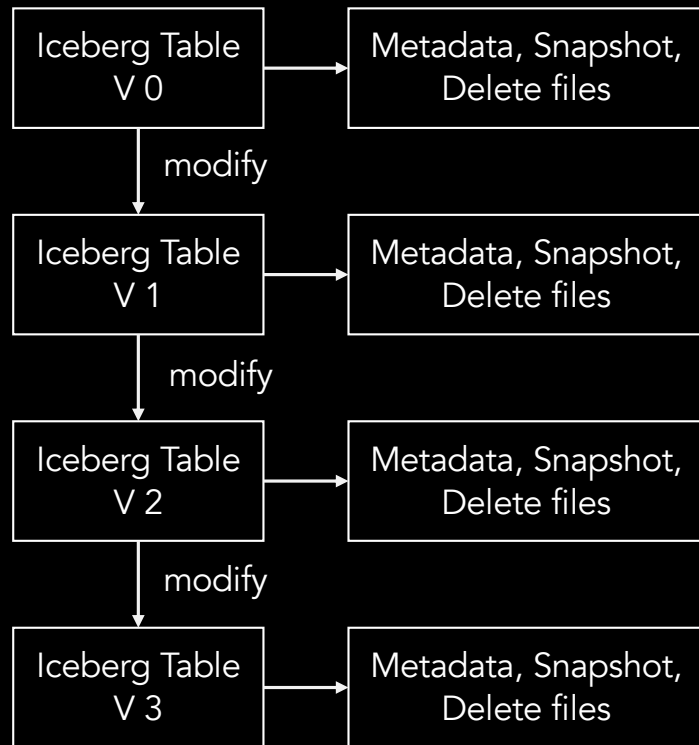
# Iceberg and DataFrame APIs

- Write
  - `df.writeTo('<iceberg-table-name>').append()` – similar to INSERT INTO ...
  - `df.writeTo('<iceberg-table-name>').create()` – similar to CREATE TABLE AS SELECT ...
  - `df.writeTo('<iceberg-table-name>').overwrite()` – similar to truncate and load.
- Create table – `data.writeTo("db.table").using("iceberg").create()`

# COW or MOR – Which One?



# Time Travel

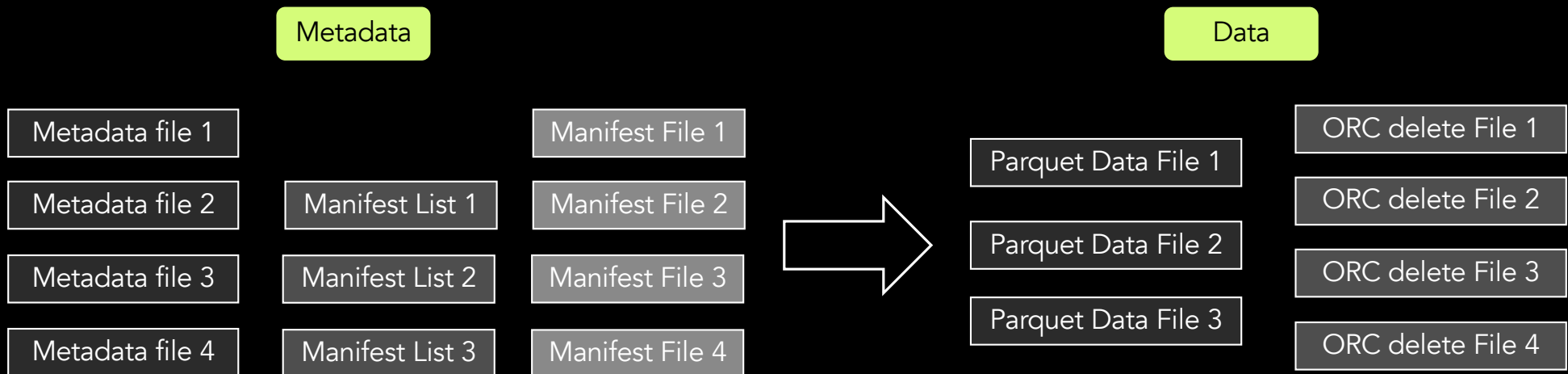


- Query table as of certain time is past.
- Supported by Iceberg snapshots.
- `SELECT ... <table> TIMESTAMP AS OF 'yyyy-mm-dd hh:mm:ss'`
- `SELECT ... FROM <table>`
- `SELECT ... <table> TIMESTAMP AS OF 'yyyy-mm-dd hh:mm:ss'`

# Get Iceberg Table Details

- `select * from <db>.<table>.history;` - Snapshot history.
- `select * from <db>.<table>.metadata_log_entries;` - Metadata logs.
- `select * from <db>.<table>.snapshots;` - All available snapshots.
- `select * from <db>.<table>.entries;` - Manifest entries;
- `select * from <db>.<table>.files;` - Data file details.
- `select * from <db>.<table>.manifest;` - File manifests.
- `select * from <db>.<table>.partitions;` - All current partition details for the table.
- `select * from <db>.<table>.all_data_files;`
- `select * from <db>.<table>.all_delete_files;`
- `select * from <db>.<table>.all_entries;`
- `select * from <db>.<table>.all_manifests;`

# Maintenance



- Each modification adds new metadata, snapshot, delete files.
- Snapshots are used by time travel queries.
- Lot of metadata files.
- Lot of delete files for MOR tables and orphaned (marked-delete) data files for COW tables.



# Table Maintenance – Snapshots

- Snapshot expiration – Deletes snapshots and corresponding data files that are not needed anymore. Expired snapshots are not available for time travel queries.
  - `CALL my_catalog.system.expire_snapshots('<database>.<table>', TIMESTAMP '<timestamp-value>', <no-of-previous-snapshots-to-keep>)`
  - `<table>.expireSnapshots().expireOlderThan(<time-in-ms>).commit()`
  - `SparkActions.get().expireSnapshots(<table>).expireOlderThan(ts).execute()`
- Table Properties
  - `history.expire.max-snapshot-age-ms` – Max age of snapshots to keep for the table.
  - `history.expire.min-snapshots-to-keep` – Min no. of snapshots to keep for the table.

# Table Maintenance – Metadata Files

- Metadata files – Remove orphaned (delete-marked) metadata files that are not needed.
  - `CALL my_catalog.system.remove_orphan_files('<database>.<table>', TIMESTAMP '<timestamp-value>', '<s3-location-of-table>')`
- Compact Iceberg tables
  - `CALL my_catalog.system.rewrite_data_files(table => "3aayaam_iceberg_db.flight_data", strategy => "binpack", options => map("min-file-size-bytes", "10000000", "remove-dangling-deletes", "true"))'.show()`
- Table Properties
  - `write.metadata.delete-after-commit.enabled` – true or false.
  - `write.metadata.previous-versions-max` – Count of max metadata files from current version.

# Table Maintenance – Vacuum, Optimize

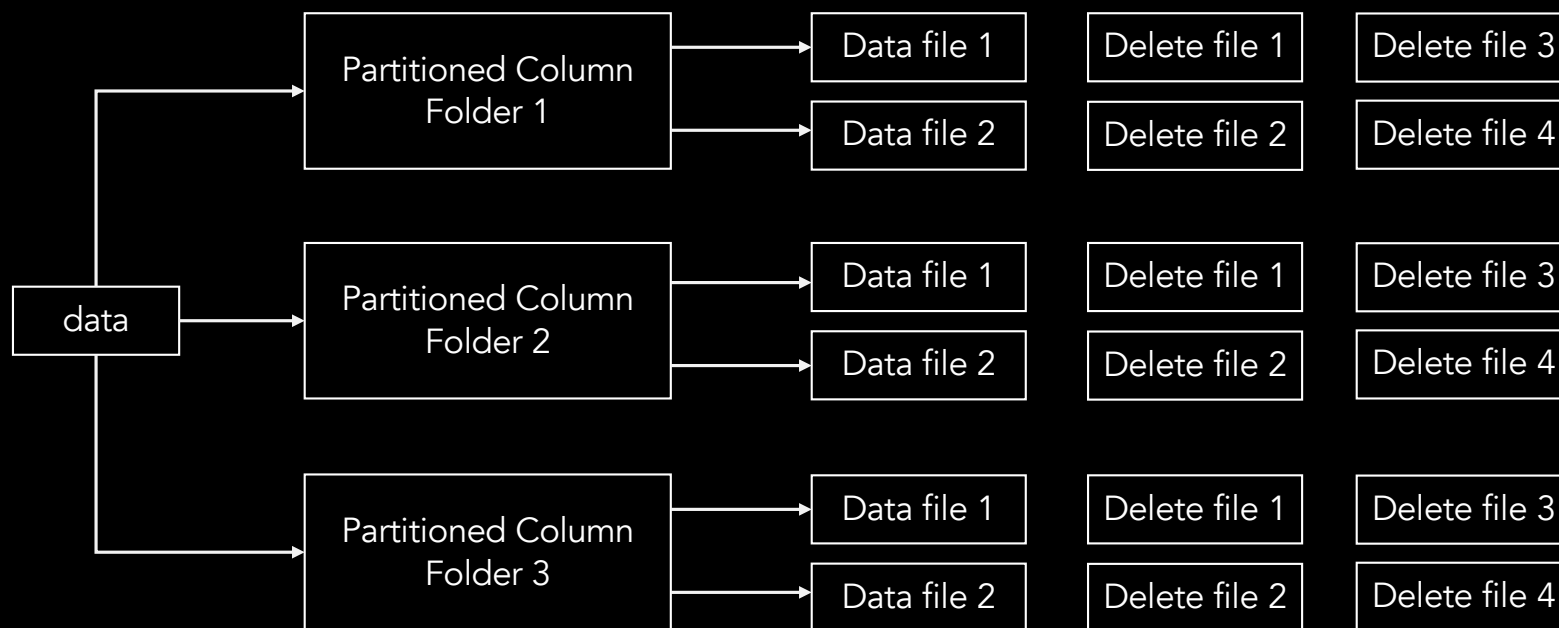
- Optimizes rows in an Iceberg table by rewriting data files into a more optimized layout based on the size and number of associated delete files.
  - `OPTIMIZE TABLE <table-name>`
- Vacuum expires snapshots and removes unused files. Reduces storage consumption.
  - `VACUUM <table-name>`

# Maintenance Operations

- `rollback_to_snapshot` – Rollback a table to a previous snapshot id.
- `rollback_to_timestamp` – Rollback a table to a previous timestamp.
- `migrate` – Migrate Parquet, Avro, Orc tables to Iceberg tables by replacing existing table. The source table will be replaced by Iceberg table.
- `snapshot` – Migrate Parquet, Avro, Orc tables to Iceberg tables by copying existing table's data. Should be created as a temporary copy for testing.
- `add_files` – Migrate by directly adding files from source tables (parquet, avro, orc) to Iceberg tables. Metadata will be added but data files will not be moved.

# Partitioned Tables

- CREATE TABLE ... USING ICEBERG PARTITIONED BY (col1, col2, ...)
- partitioned by (day(col), year(col), month(col), ... ) – Only available for Timestamp data type.



# Hidden Partitioning

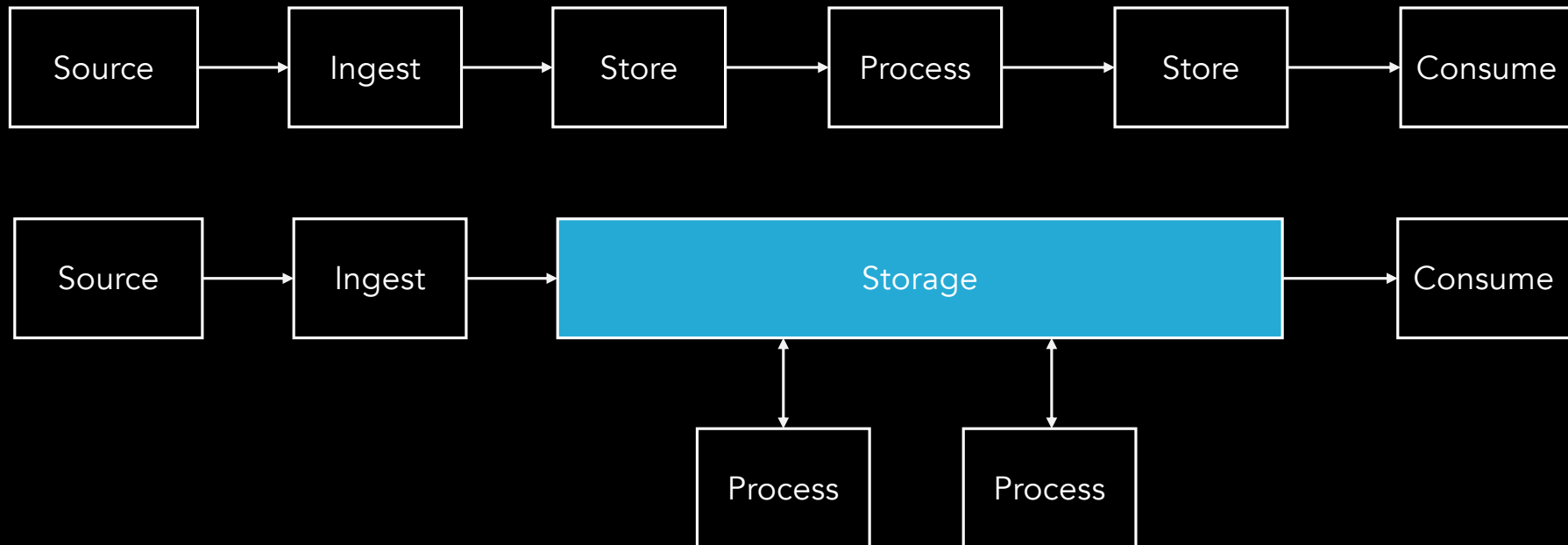
- Creates partitions using expressions/transformations.
- Partition transformations
  - `year(timestamp_col)` – Partition by year.
  - `month(timestamp_col)` – Partition by month for monthly reports.
  - `day(col)` – Partition by day for daily reports and analytics.
  - `hour(col)` – Partition by hour for operational analytics.
  - `identity(col)` – Useful for low cardinality columns.
  - `bucket(n, col)` – Column value is hashed and put to one of the n partitions. For high cardinality columns.
- CREATE and ALTER TABLE can be used for partitioning table.

# Partition Evolution

- Change from year to month, month to day etc.
- It does not impact the existing data.
- Old data will remain the same, new data will use the new partition scheme. Old partition structure can also be used.

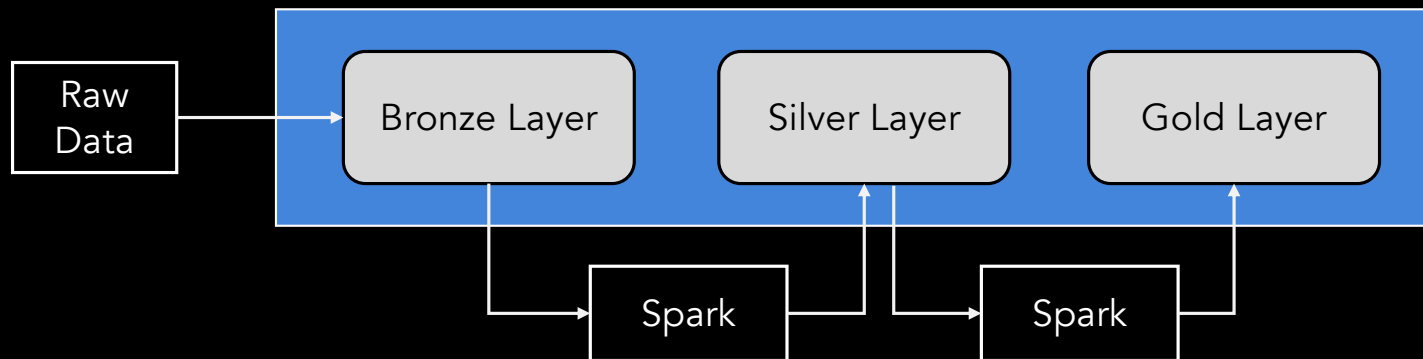
# Medallion Architecture

- How data is stored and managed for data platform.
- The idea is to improve the quality of data progressively.





# Medallion Architecture



## Advantages

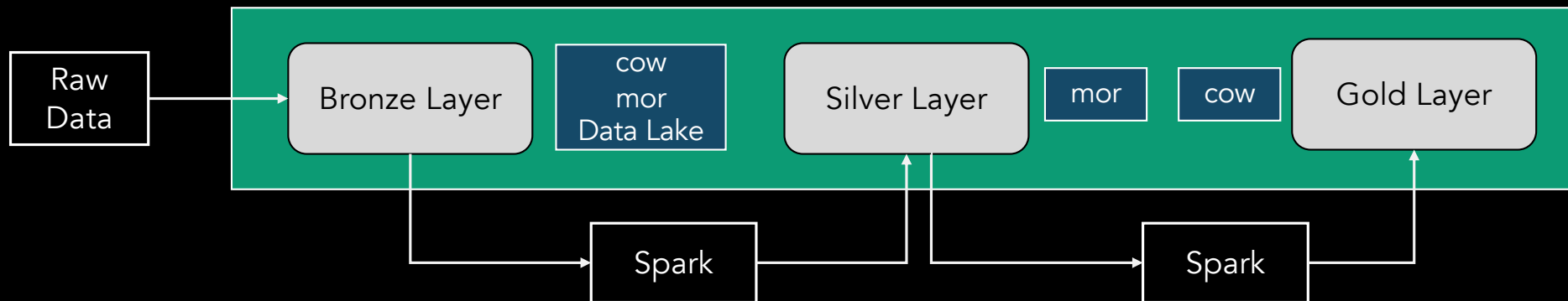
- Decoupling of layers.
- Raw data is not destroyed, recreate downstream tables anytime.
- Scalable and simpler data model.

- Unprocessed Data.
- Original format of data is preserved.
- No transformation is applied here.

- Transformation.
- Clean data.
- Deduplication.
- Enrichment.
- Self-service Analytics, ad-hoc reporting, ML.
- Write-intensive.

- Transformation.
- Modelled and processed for reporting.
- De-normalized, Star, Snowflake schema, data marts, data warehouses.
- Read intensive workload uses this layer.
- Business specific use cases.
- DSc, Advanced analytics, ML, Ad-hoc analytics, APIs etc.

# Lakehouse Architecture



- Data Lake and Data Warehouse.
- Supports ACID transactions.
- Time travel queries.
- Cost effective.

- What is Lakehouse
- Iceberg metadata layer
- Setup EMR and Jupyter Notebook with Iceberg
- Table operations, properties & maintenance
- COW, MOR tables
- Time travel, Hidden partitioning
- Medallion & Lakehouse architecture

# Summary

[www.3aayaam.in](http://www.3aayaam.in)



Thank You  
[www.3aayaam.in](http://www.3aayaam.in)